

LAB2. Upravljanje sustavom kroz periodičke prekide

Mnogi sustavi s jednostavnim periodičkim poslovima mogu se upravljati tako da sat periodički izaziva prekide, a u svakoj obradi (u svakoj periodi) obavi se samo jedan zadatak. Očekivano, zadatak mora završiti prije idućeg prekida. U protivnom, taj novi prekid prekida izvođenje prethodnog zadatka, što pomaže u otklanjanju problema ukoliko izvođenje pojedinog zadatka zapne. Na taj način pojedini zadatak neće kompromitirati ostatak sustava. Druga mogućnost je da se dozvoli produženje obrade, ali npr. najviše za jednu periodu.

Prekinuti zadatak se uobičajeno ne nastavlja nakon započinjanja i dovršetka obrade novog zadatka.

Osnovna skica takva upravljača je prikazana kodom:

```
obradi_zadatak {
    x = odaberi zadatak za obradu prema tablicama;
    obradi zadatak x
}
```

U ovoj vježbi treba simulirati ovakav način upravljanja (pokretanja sustava zadataka). U simulaciji, radi praćenja stanja i "normalnog" završetka obrade, definirati strukturu podataka koja će za svaki zadatak pratiti je li u obradi, je li prekinut i sl.

Ako zadatak nije gotov u jednoj periodi, dopustiti mu da završi u drugoj, ali samo ako mu je to druga perioda te u prethodnih 10 perioda nije bilo produljenja. U protivnom prekinuti izvođenje takvog zadatka i nastaviti sa sljedećim. Funkciju za obradu pozivati na periodički "prekid" (signal ili slično). Odabrani mehanizam MORA omogućiti novi poziv dok je stari u obradi. Ako se koristi signal (preporuka), ne zaboraviti dozvoliti prekidanje u funkciji za obradu signala (npr. sa pthread_sigmask ili postavljanjem zastavice SA_NODEFER pri registraciji funkcije sa sigaction).

Pseudokod takvog upravljača bi mogao biti kao u nastavku.

```
obradi_zadatak()
{
    ako je neki zadatak još u obradi {
        //prekidamo ovaj zadatak ili ne?
        ako je zadatak u obradi potrošio samo jednu periodu te
            nije bilo prekoračenja u prošlih 10 perioda
        {
            zabilježi da je dopuštena druga perioda zadatku
            vrati se iz ove funkcije //nastavi prethodni zadatak
        }
        inače {
            zabilježi da je obrada prekinuta
            prekini trenutnu obradu
            //"prekini" nekako ostvariti varijablama u kojima piše koji je zadatak u obradi
            //u obradi se periodički provjerava je li taj zadatak još uvijek aktualan
            //ako nije izlazi iz petlje i završava

            //započni sljedeći zadatak kodom ispod
        }
    }

    odaberi idući zadatak za obradu prema tablici
    ako je ovo je prazan period ili je taj zadatak popuna
        vrati se iz ove funkcije

    zabilježi trenutak reakcije

    dok je ovaj zadatak nije gotov i nije prekinut
        simuliraj djelić obrade ulaza "i" (npr. sleep 5 ms)

    ako je ovaj zadatak gotov (nije prekinut)
        označi da je obrada gotova
}
```

Zadatke odabirati prema tablicama, npr. slično kao u primjeru iz skripte 4.21.

```
zadA[] = {1, 2, 3}
zadB[] = {4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18}
zadC[] = {18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37}
zad[] = {zadA, zadB, zadC}
red[] = {0, 1, -1, 0, 1, -1, 0, 1, 2, -1} //za svaki prekid svakih 100 ms
t = 0
ind[] = {0, 0, 0}
MAXTIP[] = {3, 15, 20}
```

```
daj_iduci() {  
    sljedeci_zadatak = 0  
    tip = red[t]  
    t = (t + 1) mod 10  
    ako je tip != -1 tada  
        sljedeci_zadatak = zad[tip][ind[tip]]  
        ind[tip] = (ind[tip] + 1) mod MAXTIP[tip]  
    vrati sljedeci_zadatak - 1 //-1 da bude indeks polja, kreće od nule  
}
```

Simulacija ulaza treba biti ista kao i u zadatku LAB1.

Obzirom da je ovdje upravljaču statički definirano kada koji zadatak obrađuje (kroz tablice) ulazi moraju također imati ista vremenska svojstva, tj. ako neki ulaz provjerava u trenutku x, onda dretva koja simulira taj ulaz mora taj ulaz postaviti samo malo prije toga. Inače bi odstupanja bila nepotrebno veća (nepotrebno jer se ovdje upravljač optimira za taj skup ulaza). Stoga malo zakasniti s pokretanjem periodičkog prekida da simulatori ulaza ipak promijene stanje prije nego li upravljač dođe do ovog ulaza. Npr. nakon stvaranja dretvi za simulaciju ulaza spavati 10 ms i tek tada pokrenuti generiranje prekida svakih 100 ms (od tada, s 10 ms kašnjenja prema simulatorima ulaza).

Na kraju simulacije ispisati statistiku (koja sada uključuje i broj zadataka koji su koristili dvije periode i broj zadataka koji se nisu obavili do kraja).